

OSDBMS-V-2026-T038

WORK BREAKDOWN & DIVISION OF LABOR

WALnut

A Concurrent Auction Engine with Deadlock Detection & Recovery

Team: The Acid Mechanics – T038

Vinayak Suyal · Sushmit Singh Rawat · Kanak Rawat · Suhavi Jugran

Version 1.0 · Companion to the WALnut PRD, Architecture Doc, API Spec & Test Plan
Status: Draft for Review

1. Introduction

This document assigns concrete, week-by-week ownership across the four team members, building on the primary-subsystem ownership already set out in the PRD (§11) and the testing ownership in the Test Plan (§3). Its purpose is to make it obvious, at any point in the eight weeks, exactly who is building what, who each person is blocked on, and who reviews or tests whose work.

Ownership below is primary, not exclusive — every member is expected to understand the whole architecture (PRD §11) and pairs up where subsystems interlock (Section 5).

2. Roles & Primary Subsystem Ownership

Member	Primary Subsystem	Also Owns (from Test Plan)
Vinayak Suyal (Lead)	Storage manager, page format, buffer pool, page replacement policy	TC-STO-*, TC-BUF-* test cases; final PRD/Architecture doc consistency review
Sushmit Singh Rawat	Transaction manager, lock manager, Strict 2PL, wait-for-graph deadlock detector	TC-LCK-*, TC-DLK-* test cases
Kanak Rawat	Write-Ahead Logging, recovery manager, hash index, test workloads	TC-REC-* test cases; fault-injection harness; requirements traceability matrix
Suhavi Jugran	Fixed operation API, embedded REST server, dashboard, performance visualization	TC-API-*, TC-UI-* test cases; API Specification document owner

3. Week-by-Week Task Assignment

Each row corresponds to the milestone of the same week in the PRD (§10) and the acceptance gate in the Test Plan (§14).

Week 1 — Design Freeze

Member	Tasks
Vinayak	Own the high-level architecture diagram; draft the on-disk page format spec.
Sushmit	Draft the lock table structure and Strict 2PL protocol notes that will drive the lock manager.
Kanak	Finalize the Users / Items / Bids schema, ER diagram, and functional dependencies; draft the WAL record format.
Suhavi	Finalize the fixed operation API signatures (createUser, createItem, placeBid, getHighestBid, listBids, closeAuction, reserveBundle); low-fidelity dashboard wireframes.

Week 2 — Storage Foundation

Member	Tasks
Vinayak	Lead: implement the page format, file manager, record layout, and database initialization (walnut.db).

Member	Tasks
Sushmit	Stub an in-memory lock table (no persistence yet) against the finalized schema.
Kanak	Stand up the walnut.wal file format and a minimal append-only writer.
Suhavi	Scaffold the embedded REST server (cpp-httplib) and the dashboard project (React + TypeScript + Vite).

Week 3 — Buffer & Indexing

Member	Tasks
Vinayak	Lead: implement the buffer pool — pin/unpin, dirty-page tracking, Clock/LRU replacement.
Kanak	Lead: implement the hash index on item_id.
Sushmit	Integrate buffer-pool page access hooks into the transaction manager skeleton.
Suhavi	Wire read-only endpoints (getHighestBid, listBids) against the storage/index stubs.

Week 4 — Single-threaded Operations

Member	Tasks
Suhavi	Lead: wire all fixed operations end-to-end through the REST API, single-threaded.
Vinayak	Support storage/buffer-pool calls needed by each operation; fix integration bugs as they surface.
Kanak	Support index lookups needed by each operation; begin the fault-injection test harness design.
Sushmit	Build single-threaded BEGIN/COMMIT transaction lifecycle scaffolding (no concurrency yet).

Week 5 — Concurrency Core (MVP)

Member	Tasks
Sushmit	Lead: POSIX thread pool, lock manager, Strict 2PL — make placeBid fully atomic under concurrency.
Vinayak	Make the buffer pool thread-safe under concurrent pin/unpin from multiple transactions.
Kanak	Hook the write-ahead rule into the commit path (log before flush).
Suhavi	Expose concurrent placeBid over the API; build the dashboard's manual "Place Bid" control (FR-11) and bidder selector.

MVP gate (PRD §13): persistent engine safely processing concurrent bids. Whole team runs TC-LCK-01–04 together before declaring the gate met.

Week 6 — Deadlock Handling

Member	Tasks
Sushmit	Lead: wait-for graph, cycle detection, victim selection, rollback, automatic retry.
Kanak	Build the reserveBundle test harness that deterministically produces the mixed-order deadlock scenario.
Vinayak	Debug lock/buffer-pool interplay under the deadlock test scenario.

Member	Tasks
Suhavi	Wire the dashboard's live wait-for-graph visualization and deadlock.detected / deadlock.resolved SSE events.

Week 7 — Recovery

Member	Tasks
Kanak	Lead: full WAL (commit/abort records, checkpointing) and the fault-injection recovery tests.
Vinayak	Confirm buffer-pool flush ordering strictly respects the write-ahead rule under fault injection.
Sushmit	Ensure abort/rollback paths correctly trigger WAL undo for victim and manually-aborted transactions.
Suhavi	Wire the dashboard's WAL-activity view (wal.write, checkpoint.complete events).

Week 8 — Integration & Demo

Member	Tasks
Suhavi	Lead: finish the embedded REST API, complete the dashboard, finalize SSE wiring end-to-end.
Vinayak	Run and report buffer-pool benchmarks (hit rate vs. frame count); final Architecture Doc pass.
Sushmit	Run and report concurrency benchmarks (latency, throughput, deadlock frequency).
Kanak	Run and report recovery benchmarks (recovery time); finalize the requirements traceability matrix.
Whole team	Final documentation pass, demo script rehearsal, Q&A prep.

4. Testing Ownership

Mirrors the Test Plan's traceability matrix (§5) and level-ownership table (§3); each member writes and maintains the tests for their own subsystem, but any member may run any test suite.

Test Category	Primary Owner	Test IDs
Storage & Buffer Pool	Vinayak	TC-STO-01-03, TC-BUF-01-04
Concurrency & Lock Manager	Sushmit	TC-LCK-01-04
Deadlock Detection	Sushmit (impl.) + Kanak (harness)	TC-DLK-01-04
WAL & Recovery	Kanak	TC-REC-01-04
API Contract	Suhavi	TC-API-01-04
Dashboard (manual/exploratory)	Suhavi	TC-UI-01-03
Performance benchmarks	Owning member per metric (§3 above)	TC-PERF-01-03

5. Documentation Ownership

Document	Primary Author	Notes
PRD	Whole team, final edit by Vinayak (Lead)	Living document; update if scope shifts.
Architecture & Design Doc	Vinayak (storage/buffer) + Sushmit (concurrency/deadlock) + Kanak (WAL section)	Each contributes their own section; Vinayak assembles.
API Specification	Suhavi	Kept in sync with the actual REST server — update this doc in the same PR as any endpoint change.
Test Plan	Kanak assembles; each member supplies their own subsystem's test cases	Kanak also owns the requirements traceability matrix.
README	Suhavi (build/run steps) + whole team (milestone status)	Update the milestone checklist at each weekly gate.

6. Cross-Team Dependencies & Handoff Points

- Sushmit's lock manager (Week 5) depends on Vinayak's buffer pool exposing thread-safe pin/unpin by end of Week 3.
- Kanak's WAL/recovery (Week 7) depends on Vinayak's buffer pool honoring the write-ahead rule hook added in Week 5, and on Sushmit's transaction manager correctly reporting commit/abort boundaries.
- Suhavi's API layer (Week 4 onward) depends on Vinayak's storage manager and Kanak's hash index being callable — even as stubs — by end of Week 3.
- Suhavi's dashboard deadlock visualization (Week 6) depends on Sushmit's wait-for graph emitting lock.waiting / deadlock.detected events in the exact shape defined in the API Specification §6.1.
- Kanak's reserveBundle deadlock test harness (Week 6) depends on Suhavi's reserveBundle endpoint and Sushmit's multi-lock acquisition both being in place.

Any change to a shared interface (lock table shape, WAL record format, REST/SSE payloads) must be reflected in the API Specification or Architecture Doc in the same week it changes — per the versioning note in the API Specification §8.

7. Collaboration Cadence

- A short sync at the start of each week to confirm the previous week's gate was met (per Test Plan §14) before starting the next week's tasks.
- Any blocking dependency (Section 6) is raised as soon as it's known to be at risk — not at the end of the week.
- Code review: each subsystem's primary owner reviews changes to their own area; cross-cutting changes (shared headers, schema, API contracts) get a second reviewer from an adjacent subsystem.
- All defects logged and triaged per the Test Plan's Defect Management process (§15).